

13-flutter-mobile

13. Flutter Mobile — Dari Dart ke Play Store

Level: ✗ Intermediate

Jam: 8 (4 minggu × 2 sesi)

Prasyarat: TypeScript/JavaScript dasar, OOP

Output: Flutter app siap deploy (Android APK ke Play Store)

Kenapa Flutter?

Flutter adalah framework UI **open-source** dari Google buat bikin aplikasi mobile, web, dan desktop dari **satu codebase**. Bahasa yang dipake: **Dart**.

Flutter paling besar di Indonesia — buktinya:

- **Gojek, Tokopedia, BCA Mobile, Lapak Gaming, Ralali** pake Flutter.
- Lebih dari **40% developer mobile** Indonesia pake Flutter (Survey IDS 2024).
- **Startup dan agensi** di Indonesia dominan Flutter karena biaya lebih murah (satu tim, dua platform: Android + iOS).
- **Komunitas Flutter Indonesia** aktif banget — Telegram, Discord, meetup tiap bulan.

Kalau lo udah bisa JavaScript/TypeScript, lo udah bekal. Tinggal belajar **sintaks Dart** dan mental **widget tree**.

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Nulis kode Dart sendiri — variable, function, class, async

- Pahami Widget Tree dan beda StatelessWidget vs StatefulWidget
 - Layout pake Row, Column, ListView, GridView
 - Pake setState dan Provider buat state management
 - Navigasi antar screen pake Navigator & named routes
 - Panggil REST API + parse JSON pake http package
 - Setup Firebase (Firestore + Auth)
 - Build APK dan deploy ke Play Store
-

Materi

Sesi	Topik	File
1	Dart Syntax & Widget Dasar	01-dart-widget.md
2	Layout & State Management	02-layout-state.md
3	Navigation & API Integration	03-navigation-api.md
4	Firebase & Deploy ke Play Store	04-firebase-deploy.md

Output Akhir Modul

Flutter app siap deploy — aplikasi mobile dengan Firebase backend, upload APK ke Google Play Console.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Explain this Flutter widget tree line by line"
 - "Refactor this StatefulWidget to use Provider"
 - "Convert this API call to use FutureBuilder"
 - "Generate 3 test cases for this Dart function"
 - "Find the bug in this widget state management code"
 - "Convert this Firebase Firestore query to a paginated version"
-

Rangkuman

Konsep	Poin Penting
Dart	Sintaks mirip TS, null safety default, final/const
Widget	Stateless (statis) vs Stateful (dinamis), setState() trigger rebuild
Layout	Row/Column/Expanded, ListView/GridView, EdgeInsets/Padding
Navigasi	push/pop, named routes, onGenerateRoute
State Management	setState buat lokal, Provider buat global
API	http package, dart:convert, FutureBuilder
Firebase	Firestore CRUD, Auth login/register, security rules
Deploy	Keystore → key.properties → flutter build appbundle → Play Console

Referensi

- [Flutter Documentation](#) — resmi dari Google
- [Dart Null Safety](#) — wajib baca
- [Provider Package](#) — state management
- [Firebase Flutter Setup](#) — integrasi Firebase
- [Komunitas Flutter Indonesia](#) — tanya-tanya di Telegram
- [Google Play Console](#) — deploy app

SMK Bisa! Dengan Flutter, lo bisa bikin app Android + iOS sendirian. Modal laptop, koneksi internet, dan konsisten ngoding tiap hari. Mulai dari widget sederhana, API call, sampe publish ke Play Store.

1.1 Dart Syntax & Widget Dasar

Variabel & Tipe Data

```
// Type inference
var nama = 'Budi';           // → String
var umur = 17;               // → int
var suhu = 36.5;             // → double
var aktif = true;            // → bool

// Eksplisit
String nama2 = 'Sari';
int umur2 = 17;
double suhu2 = 36.5;
bool aktif2 = true;
```

```
// Final = sekali assign (runtime constant)
final apiKey = 'rahasia123';
// apiKey = 'baru'; // ERROR!

// Const = compile-time constant
const pi = 3.14;
const kuadratPi = pi * pi;

// Null safety – WAJIB paham!
String? alamat; // nullable → bisa null
String kota = 'Jakarta'; // non-null → error kalo diisi null
// kota = null; // COMPILE ERROR!
```

Null safety: Dart beda sama JS/TS. String artinya **never null**. Kalo lo butuh null, pake String?. Ini ngilangin null pointer error dari awal.

Function

```
// Regular function
int tambah(int a, int b) {
  return a + b;
}

// Expression body (arrow) – kaya JS arrow function
int kali(int a, int b) => a * b;

// Named parameter (wajib {})
void sapa({required String nama, int? umur}) {
  print('Halo $nama');
  if (umur != null) print('Umur $umur');
}

// Panggil: sapa(nama: 'Budi', umur: 17)

// Optional positional parameter
void greet(String nama, [String? gelar]) {
  if (gelar != null) {
    print('Halo $gelar $nama');
  } else {
    print('Halo $nama');
  }
}

// Higher-order function (callback)
void prosesList(List<int> angka, Function(int) callback) {
```

```

    for (var a in angka) {
        callback(a);
    }
}

```

Class

```

// Constructor ringkas pake this.
class User {
    final String name;
    final int age;
    final String? email;

    // Constructor: langsung assign
    User({required this.name, required this.age, this.email});

    // Method
    void greet() => print('Halo, saya $name');

    // Named constructor
    User.admin(String name, {this.email})
        : name = name,
          age = 18;

    // Factory constructor
    factory User.fromMap(Map<String, dynamic> map) {
        return User(
            name: map['name'] as String,
            age: map['age'] as int,
            email: map['email'] as String?,
        );
    }
}

// Pake
void main() {
    final user = User(name: 'Budi', age: 17);
    user.greet(); // Halo, saya Budi

    final admin = User.admin('Admin', email: 'admin@test.com');
    print(admin.age); // 18
}

```

Async / Await

```
// Future = Promise di JS
Future<String> fetchData() async {
  await Future.delayed(Duration(seconds: 2)); // simulasi delay
  return 'Data dari server';
}

// Async function return Future
Future<void> main() async {
  print('Mulai loading...');
  final data = await fetchData();
  print(data); // Data dari server
  print('Selesai');
}

// Error handling
Future<void> getData() async {
  try {
    final data = await fetchData();
    print(data);
  } catch (e) {
    print('Error: $e');
  } finally {
    print('Selesai');
  }
}
```

Widget Tree

Di Flutter, **semua adalah Widget**. Layar, tombol, teks, padding — semuanya widget. Widget disusun **nested** jadi tree.

```
MaterialApp
├── Scaffold
│   ├── AppBar
│   │   └── Text (judul)
│   └── Body
│       ├── Center
│       │   └── Column
│       │       ├── Text
│       │       ├── SizedBox (jarak)
│       │       └── ElevatedButton
```

StatelessWidget

Widget yang **tidak berubah** setelah di-render.

```
import 'package:flutter/material.dart';

class ProfileCard extends StatelessWidget {
  final String name;
  final int age;

  const ProfileCard({super.key, required this.name, required this.age});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Text(name, style: const TextStyle(fontSize: 20)),
            const SizedBox(height: 8),
            Text('$age tahun'),
          ],
        ),
      ),
    );
  }
}
```

StatefulWidget

Widget yang **bisa berubah** — tombol counter, input form, timer.

```
class CounterScreen extends StatefulWidget {
  const CounterScreen({super.key});

  @override
  State<CounterScreen> createState() => _CounterScreenState();
}

class _CounterScreenState extends State<CounterScreen> {
  int _count = 0; // state internal

  void _increment() {
    setState(() {
      _count++;
    }); // ← trigger build ulang
  }
}
```

```

    }

    void _reset() {
      setState(() => _count = 0);
    }

    @override
    Widget build(BuildContext context) {
      return Scaffold(
        appBar: AppBar(title: const Text('Counter')),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const Text('Kamu tekan tombol:'),
              Text('$_count',
                style: const TextStyle(fontSize: 48, fontWeight: FontWeight.bold),
                const SizedBox(height: 20),
                ElevatedButton(
                  onPressed: _reset,
                  child: const Text('Reset'),
                ),
              ],
            ),
          ),
        floatingActionButton: FloatingActionButton(
          onPressed: _increment,
          child: const Icon(Icons.add),
        ),
      );
    }
  }
}

```

Pola penting: State dipisah dari Widget. Widgetnya immutable, Statenya yang pegang data. `setState()` kasih tau Flutter "bangun ulang widget-ku".

MaterialApp & Scaffold

```

import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {

```



```

const MyApp({super.key});

@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Aplikasi Pertama',
    theme: ThemeData(
      colorSchemeSeed: Colors.blue,
      useMaterial3: true,
    ),
    home: const HomeScreen(),
  );
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Home'),
        centerTitle: true,
      ),
      body: const Center(
        child: Text('Hello Flutter!', style: TextStyle(fontSize: 24)),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          print('Tombol ditekan!');
        },
        child: const Icon(Icons.star),
      ),
    );
  }
}

```

Widget Dasar Lainnya

```

// Text – teks dengan style
Text('Halo Dunia',
  style: TextStyle(fontSize: 18, color: Colors.blue, fontWeight: FontWeight.bold),
)

// Container – box dengan padding, margin, decoration

```

```

Container(
  padding: EdgeInsets.all(16),
  margin: EdgeInsets.symmetric(horizontal: 8, vertical: 4),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(12),
    boxShadow: [
      BoxShadow(color: Colors.black26, blurRadius: 4),
    ],
  ),
  child: Text('Ini container'),
);

// SizedBox – spacer
SizedBox(width: 16, height: 16); // kotak transparan
SizedBox(height: 20); // vertical spacer

// Row & Column – sudah di sesi 2

```

Latihan

1. **Buat ProfileCard Widget** — Widget StatelessWidget yang menerima name, nim, kelas. Tampilkan pake Card + ListTile.
2. **Buat Counter App** — Di atas ada CounterScreen. Tambahkan tombol Decrement (-) dan tombol reset. Bonus: jangan biarin counter minus (min 0).
3. **Buat App Sederhana** — MaterialApp + Scaffold + AppBar + body Center + Column. Isinya: Text nama lo, SizedBox, Text kelas lo, SizedBox, ElevatedButton yang print "Halo!" ke console.
4. **Bongkar Null Safety** — Bikin class Product dengan field name (String non-null), price (double non-null), description (String? nullable). Bikin constructor + method printInfo() yang ngeprint "Produk: [name] — Rp[price]" dan kalo description ada, print juga description.

1.2 Layout & State Management

Layout Widgets

Row — horizontal

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceEvenly, // horizontal  
  crossAxisAlignment: CrossAxisAlignment.center,    // vertical  
  children: [  
    Icon(Icons.star, color: Colors.yellow, size: 40),  
    Icon(Icons.star, color: Colors.yellow, size: 40),  
    Icon(Icons.star, color: Colors.yellow, size: 40),  
  ],  
);
```

Column — vertical

```
Column(  
  mainAxisAlignment: MainAxisAlignment.center,    // vertical  
  crossAxisAlignment: CrossAxisAlignment.start,  // horizontal  
  children: [  
    Text('Nama: Budi'),  
    Text('Kelas: XII RPL'),  
    Text('Sekolah: SMK Negeri 1'),  
  ],  
);
```

Expanded — isi sisa ruang

```
Row(  
  children: [  
    Expanded(  
      flex: 2, // 2 bagian  
      child: Container(color: Colors.red),  
    ),  
    Expanded(  
      flex: 1, // 1 bagian  
      child: Container(color: Colors.blue),  
    ),  
  ],  
);  
// Rasio: merah 2/3, biru 1/3
```

Flexible vs Expanded

```
Row(  
  children: [  
    Container(width: 50, height: 50, color: Colors.red),  
    Flexible(  
      fit: FlexFit.loose, // bisa lebih kecil dari child  
      child: Container(height: 50, color: Colors.green),  
    ),  
    Expanded(  
      // fit: FlexFit.tight – WAJIB isi ruang  
      child: Container(height: 50, color: Colors.blue),  
    ),  
  ],  
);
```

Padding & Margin

```
// EdgeInsets – semua cara  
EdgeInsets.all(16); // semua sisi  
EdgeInsets.symmetric(horizontal: 16, vertical: 8); // horizontal + vertical  
EdgeInsets.only(left: 16, top: 8); // spesifik sisi  
  
// Pake di widget  
Padding(  
  padding: EdgeInsets.all(16),  
  child: Text('Hello'),  
);  
  
Container(  
  margin: EdgeInsets.symmetric(vertical: 8),  
  padding: EdgeInsets.all(12),  
  child: Text('Container dengan margin + padding'),  
);
```

ListView — scrollable list

```
ListView(  
  children: [  
    ListTile(title: Text('Item 1'), subtitle: Text('Deskripsi')),  
    ListTile(title: Text('Item 2'), trailing: Icon(Icons.arrow_forward)),  
    ListTile(title: Text('Item 3'), leading: Icon(Icons.person)),  
  ],  
);  
  
// ListView.builder – performa tinggi
```

```

ListView.builder(
  itemCount: 100,
  itemBuilder: (context, index) {
    return ListTile(
      leading: CircleAvatar(child: Text('$index')),
      title: Text('Item ke-$index'),
    );
  },
);

```

GridView — grid layout

```

GridView.count(
  crossAxisCount: 2, // 2 kolom
  crossAxisSpacing: 8,
  mainAxisSpacing: 8,
  padding: EdgeInsets.all(8),
  children: [
    Container(color: Colors.red, height: 100),
    Container(color: Colors.blue, height: 100),
    Container(color: Colors.green, height: 100),
    Container(color: Colors.orange, height: 100),
  ],
);

// GridView.builder — dinamis
GridView.builder(
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
    childAspectRatio: 3 / 4, // lebar : tinggi
  ),
  itemCount: products.length,
  itemBuilder: (context, index) {
    return ProductCard(product: products[index]);
  },
);

```

Layout combo — contoh profile screen

```

import 'package:flutter/material.dart';

class ProfileScreen extends StatelessWidget {
  const ProfileScreen({super.key});

  @override

```

```

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Profile')),
    body: Column(
      children: [
        // Header
        Container(
          width: double.infinity,
          padding: const EdgeInsets.all(24),
          color: Colors.blue.shade50,
          child: Column(
            children: [
              const CircleAvatar(
                radius: 50,
                backgroundImage: NetworkImage('https://i.pravatar.cc/150'),
              ),
              const SizedBox(height: 12),
              const Text('Budi Santoso',
                style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
              Text('budi@email.com', style: TextStyle(color: Colors.grey[600]))
            ],
          ),
        ),
        // Menu list
        Expanded(
          child: ListView(
            children: [
              ListTile(leading: Icon(Icons.settings), title: Text('Pengaturan')),
              ListTile(leading: Icon(Icons.info), title: Text('Tentang')),
              ListTile(leading: Icon(Icons.logout), title: Text('Keluar')),
            ],
          ),
        ),
      ],
    ),
  );
}

```

State Management

Level 1: setState

```
class TodoScreen extends StatefulWidget {
  const TodoScreen({super.key});

  @override
  State<TodoScreen> createState() => _TodoScreenState();
}

class _TodoScreenState extends State<TodoScreen> {
  final List<String> _todos = [];
  final _controller = TextEditingController();

  void _addTodo() {
    if (_controller.text.isNotEmpty) {
      setState(() {
        _todos.add(_controller.text);
        _controller.clear();
      });
    }
  }

  void _removeTodo(int index) {
    setState(() => _todos.removeAt(index));
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Todo List')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(16),
            child: Row(
              children: [
                Expanded(
                  child: TextField(
```

```

        controller: _controller,
        decoration: const InputDecoration(
          hintText: 'Tambah todo...',
          border: OutlineInputBorder(),
        ),
      ),
    ),
    const SizedBox(width: 8),
    ElevatedButton(
      onPressed: _addTodo,
      child: const Text('Tambah'),
    ),
  ],
),
),
Expanded(
  child: ListView.builder(
    itemCount: _todos.length,
    itemBuilder: (context, index) {
      return ListTile(
        title: Text(_todos[index]),
        trailing: IconButton(
          icon: const Icon(Icons.delete, color: Colors.red),
          onPressed: () => _removeTodo(index),
        ),
      );
    },
  ),
),
),
),
);
}
}

```

StatefulWidget Lifecycle

createState()	→ bikin state object
initState()	→ inisialisasi (panggil 1x)
didChangeDependencies()	→ dependencies berubah
build()	→ render widget
setState()	→ trigger build ulang
didUpdateWidget()	→ widget parent berubah
dispose()	→ cleanup (panggil 1x)

```
class LifecycleDemo extends StatefulWidget {
```



```

const LifecycleDemo({super.key});

@override
State<LifecycleDemo> createState() => _LifecycleDemoState();
}

class _LifecycleDemoState extends State<LifecycleDemo> {
  @override
  void initState() {
    super.initState();
    print('□ initState – jalan 1x');
    // Init controller, stream subscription, API call
  }

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    print('□ didChangeDependencies – dependencies berubah');
    // Pake context.inheritFromWidgetOfExactType()
  }

  @override
  void didUpdateWidget(covariant LifecycleDemo oldWidget) {
    super.didUpdateWidget(oldWidget);
    print('□ didUpdateWidget – parent rebuild');
    // Bandingin widget lama & baru
  }

  @override
  void dispose() {
    print('□ dispose – cleanup');
    // Hapus controller, stream, listener
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    print('□ build – render');
    return const SizedBox.shrink();
  }
}

```

Level 2: Provider

Provider = state management paling populer di Flutter. Pake pattern **ChangeNotifier + Provider**.

Setup pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  provider: ^6.0.0
```

Buat Model + ChangeNotifier

```
// models/todo_provider.dart  
import 'package:flutter/foundation.dart';  
  
class TodoItem {  
  final String title;  
  bool isDone;  
  
  TodoItem({required this.title, this.isDone = false});  
}  
  
class TodoProvider extends ChangeNotifier {  
  final List<TodoItem> _items = [];  
  
  List<TodoItem> get items => _items;  
  int get doneCount => _items.where((i) => i.isDone).length;  
  int get pendingCount => _items.where((i) => !i.isDone).length;  
  
  void add(String title) {  
    _items.add(TodoItem(title: title));  
    notifyListeners(); // ← WAJIB – trigger rebuild widget  
  }  
  
  void toggle(int index) {  
    _items[index].isDone = !_items[index].isDone;  
    notifyListeners();  
  }  
  
  void remove(int index) {  
    _items.removeAt(index);  
    notifyListeners();  
  }  
}
```

Bungkus App pake Provider

```
// main.dart
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TodoProvider(),
      child: const MyApp(),
    ),
  );
}
```

Pake di Widget

```
class TodoListScreen extends StatelessWidget {
  const TodoListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    // watch = listen perubahan → rebuild
    final todoProvider = context.watch<TodoProvider>();

    return Scaffold(
      appBar: AppBar(title: const Text('Todo with Provider')),
      body: Column(
        children: [
          // Stats
          Padding(
            padding: const EdgeInsets.all(16),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.spaceAround,
              children: [
                Text('Total: ${todoProvider.items.length}'),
                Text('Done: ${todoProvider.doneCount}'),
                Text('Pending: ${todoProvider.pendingCount}'),
              ],
            ),
          ),
          // List
          Expanded(
            child: ListView.builder(
              itemCount: todoProvider.items.length,
              itemBuilder: (context, index) {
                final item = todoProvider.items[index];
                return ListTile(
                  leading: Checkbox(
```

```

        value: item.isDone,
        onChanged: (_) => todoProvider.toggle(index),
      ),
      title: Text(
        item.title,
        style: TextStyle(
          decoration:
            item.isDone ? TextDecoration.lineThrough : null,
        ),
      ),
      trailing: IconButton(
        icon: const Icon(Icons.delete, color: Colors.red),
        onPressed: () => todoProvider.remove(index),
      ),
    );
  },
),
],
),
floatingActionButton: FloatingActionButton(
  onPressed: () => _showAddDialog(context),
  child: const Icon(Icons.add),
),
);
}

void _showAddDialog(BuildContext context) {
  final controller = TextEditingController();
  showDialog(
    context: context,
    builder: (ctx) => AlertDialog(
      title: const Text('Tambah Todo'),
      content: TextField(
        controller: controller,
        decoration: const InputDecoration(hintText: 'Nama todo...'),
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx),
          child: const Text('Batal'),
        ),
        ElevatedButton(
          onPressed: () {
            // read = panggil method, ga listen
            context.read<TodoProvider>().add(controller.text);
          },
        ),
      ],
    ),
  );
}

```

```

        Navigator.pop(ctx);
      },
      child: const Text('Tambah'),
    ),
  ],
),
);
}
}

```

Provider Patterns: watch vs read

```

// watch – listen perubahan, rebuild kalo data berubah
final provider = context.watch<TodoProvider>();
Text('Done: ${provider.doneCount}'); // auto update

// read – akses data/method, GA rebuild
context.read<TodoProvider>().add('Belajar Flutter');

```

Latihan

1. **Buat Grid Galeri** — GridView.count dengan 3 kolom. Tiap item: Container warna random (pake list warna), tinggi 120, pake margin 4. Klik item → print warna dari item ke console.
2. **Buat Form Login** — Column + 2 TextField (email, password) + ElevatedButton. Pake TextEditingController. Kalo submit, print ke console "Login: [email] / [password]". Validasi: jangan kosong.
3. **Buat Counter dengan Provider** — Pindahin logic Counter dari setState ke Provider (CounterProvider extends ChangeNotifier). Widget listen pake context.watch<CounterProvider>(). Method: increment, decrement, reset.
4. **Buat Shopping Cart** — CartProvider extends ChangeNotifier. CartItem class: name, price, quantity. Method: addItem(name, price), removeItem(index), totalPrice. UI: ListView item keranjang + total harga di bawah. Pake FloatingActionButton buat nambah item (dialog input nama & harga).

1.3 Navigation & API Integration

Navigator — Push & Pop

```
import 'package:flutter/material.dart';

// Screen A → Screen B
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (_) => const DetailScreen(itemId: '123'),
              ),
            );
          },
          child: const Text('Ke Detail'),
        ),
      ),
    );
  }
}

class DetailScreen extends StatelessWidget {
  final String itemId;
  const DetailScreen({super.key, required this.itemId});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Detail $itemId')),
      body: Center(
        child: ElevatedButton(
          onPressed: () => Navigator.pop(context), // balik
          child: const Text('Kembali'),
        ),
      ),
    );
  }
}
```

```

    }
  }
}

```

Passing Data Antar Screen

```

// Kirim data
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => ProductDetail(product: selectedProduct),
  ),
);

// Terima data balik (result)
final result = await Navigator.push<bool>(
  context,
  MaterialPageRoute(
    builder: (_) => const ConfirmScreen(message: 'Yakin hapus?'),
  ),
);
if (result == true) {
  print('User konfirmasi');
}

```

Named Routes

```

// main.dart
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (_) => const HomeScreen(),
    '/profile': (_) => const ProfileScreen(),
    '/settings': (_) => const SettingsScreen(),
  },
);

// Navigasi
Navigator.pushNamed(context, '/profile');

// Kirim data lewat arguments
Navigator.pushNamed(context, '/detail', arguments: userData);

// Terima arguments
// Di DetailScreen:
final userData = ModalRoute.of(context)!.settings.arguments as User;

```

Named Route dengan Parameter Dinamis

```
// Route definition
MaterialApp(
  initialRoute: '/',
  onGenerateRoute: (settings) {
    // /product/42
    final uri = Uri.parse(settings.name!);
    final segments = uri.pathSegments;

    if (segments.length == 2 && segments[0] == 'product') {
      final id = segments[1];
      return MaterialPageRoute(
        builder: (_) => ProductScreen(productId: id),
      );
    }

    // Arguments-based
    if (settings.name == '/checkout') {
      final cart = settings.arguments as CartProvider;
      return MaterialPageRoute(
        builder: (_) => CheckoutScreen(cart: cart),
      );
    }

    // Fallback - 404
    return MaterialPageRoute(
      builder: (_) => const NotFoundScreen(),
    );
  },
);

// Navigasi
Navigator.pushNamed(context, '/product/42');
Navigator.pushNamed(context, '/checkout', arguments: cartProvider);
```

Bottom Navigation

```
class MainScreen extends StatefulWidget {
  const MainScreen({super.key});

  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
```



```

int _selectedIndex = 0;

final List<Widget> _screens = const [
  HomeTab(),
  SearchTab(),
  ProfileTab(),
];

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: _screens[_selectedIndex],
    bottomNavigationBar: NavigationBar(
      selectedIndex: _selectedIndex,
      onDestinationSelected: (index) {
        setState(() => _selectedIndex = index);
      },
      destinations: const [
        NavigationDestination(icon: Icon(Icons.home), label: 'Home'),
        NavigationDestination(icon: Icon(Icons.search), label: 'Cari'),
        NavigationDestination(icon: Icon(Icons.person), label: 'Profil'),
      ],
    ),
  );
}

```

API Integration

Setup http package

```

# pubspec.yaml
dependencies:
  http: ^1.0.0

```

Step 1: Model + JSON Serialization

```

// models/product.dart
class Product {
  final int id;
  final String name;
  final double price;
  final String? imageUrl;
}

```

```

Product({
  required this.id,
  required this.name,
  required this.price,
  this.imageUrl,
});

// JSON → Dart object
factory Product.fromJson(Map<String, dynamic> json) {
  return Product(
    id: json['id'] as int,
    name: json['name'] as String,
    price: (json['price'] as num).toDouble(),
    imageUrl: json['image_url'] as String?,
  );
}

// Dart object → JSON (buat kirim data)
Map<String, dynamic> toJson() => {
  'id': id,
  'name': name,
  'price': price,
  'image_url': imageUrl,
};
}

```

Step 2: Service Layer

```

// services/product_service.dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/product.dart';

class ProductService {
  final String baseUrl;

  ProductService({this.baseUrl = 'https://api.example.com'});

  Future<List<Product>> fetchProducts() async {
    final response = await http.get(Uri.parse('$baseUrl/products'));

    if (response.statusCode == 200) {
      final List<dynamic> data = jsonDecode(response.body) as List<dynamic>;
      return data
        .map((json) => Product.fromJson(json as Map<String, dynamic>))
    }
  }
}

```

```

        .toList();
    } else {
        throw Exception('Gagal load produk: ${response.statusCode}');
    }
}

Future<Product> fetchProduct(int id) async {
    final response = await http.get(Uri.parse('$baseUrl/products/$id'));

    if (response.statusCode == 200) {
        return Product.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
    } else {
        throw Exception('Produk tidak ditemukan: $id');
    }
}

Future<Product> createProduct(Product product) async {
    final response = await http.post(
        Uri.parse('$baseUrl/products'),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode(product.toJson()),
    );

    if (response.statusCode == 201) {
        return Product.fromJson(jsonDecode(response.body) as Map<String, dynamic>);
    } else {
        throw Exception('Gagal buat produk: ${response.statusCode}');
    }
}
}

```

Step 3: FutureBuilder di UI

```

class ProductListScreen extends StatelessWidget {
    final ProductService service = ProductService();

    ProductListScreen({super.key});

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(title: const Text('Produk')),
            body: FutureBuilder<List<Product>>(
                future: service.fetchProducts(),
                builder: (context, snapshot) {

```

```

// Loading state
if (snapshot.connectionState == ConnectionState.waiting) {
  return const Center(child: CircularProgressIndicator());
}

// Error state
if (snapshot.hasError) {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        const Icon(Icons.error_outline, size: 48, color: Colors.red),
        const SizedBox(height: 16),
        Text('Error: ${snapshot.error}',
          textAlign: TextAlign.center),
        const SizedBox(height: 16),
        ElevatedButton(
          onPressed: () {
            // Trigger rebuild → refetch
            (context as Element).reassemble();
          },
          child: const Text('Coba Lagi'),
        ),
      ],
    ),
  );
}

// Success state (data ada)
final products = snapshot.data!;
return ListView.builder(
  itemCount: products.length,
  itemBuilder: (context, index) {
    final p = products[index];
    return ListTile(
      leading: p.imageUrl != null
        ? Image.network(p.imageUrl!, width: 50, height: 50,
          fit: BoxFit.cover)
        : const CircleAvatar(child: Icon(Icons.inventory)),
      title: Text(p.name),
      subtitle: Text('Rp${p.price.toStringAsFixed(0)}'),
      trailing: const Icon(Icons.arrow_forward_ios, size: 16),
      onTap: () {
        Navigator.pushNamed(context, '/product/${p.id}');
      },
    );
  }
);

```

```

    },
  ),
),
);
}
}

```

Error Handling yang Lebih Baik

```

// services/api_result.dart
class ApiResult<T> {
  final T? data;
  final String? error;
  final bool isLoading;

  ApiResult({this.data, this.error, this.isLoading = false});

  factory ApiResult.loading() => ApiResult(isLoading: true);
  factory ApiResult.success(T data) => ApiResult(data: data);
  factory ApiResult.failure(String error) => ApiResult(error: error);
}

```

Provider + API (Best Practice)

```

// providers/product_provider.dart
import 'package:flutter/foundation.dart';
import '../models/product.dart';
import '../services/product_service.dart';

class ProductProvider extends ChangeNotifier {
  final ProductService _service = ProductService();

  List<Product> _products = [];
  bool _isLoading = false;
  String? _error;

  List<Product> get products => _products;
  bool get isLoading => _isLoading;
  String? get error => _error;

  Future<void> loadProducts() async {
    _isLoading = true;
    _error = null;
    notifyListeners();
  }
}

```

```

    try {
      _products = await _service.fetchProducts();
    } catch (e) {
      _error = e.toString();
    } finally {
      _isLoading = false;
      notifyListeners();
    }
  }
}

// UI dengan Provider
class ProductScreen extends StatelessWidget {
  const ProductScreen({super.key});

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<ProductProvider>();

    return Scaffold(
      appBar: AppBar(title: const Text('Produk')),
      body: provider.isLoading
        ? const Center(child: CircularProgressIndicator())
        : provider.error != null
          ? Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  Text('Error: ${provider.error}'),
                  ElevatedButton(
                    onPressed: () => context
                      .read<ProductProvider>()
                      .loadProducts(),
                    child: const Text('Coba Lagi'),
                  ),
                ],
              ),
            )
          : ListView.builder(
              itemCount: provider.products.length,
              itemBuilder: (context, index) {
                final p = provider.products[index];
                return ListTile(title: Text(p.name));
              },
            ),
    );
  }
}

```

```

        floatingActionButton: FloatingActionButton(
            onPressed: () =>
                context.read<ProductProvider>().loadProducts(),
            child: const Icon(Icons.refresh),
        ),
    );
}
}

```

Refresh Pake Pull-to-Refresh

```

RefreshIndicator(
    onRefresh: () => context.read<ProductProvider>().loadProducts(),
    child: ListView.builder(
        itemCount: provider.products.length,
        itemBuilder: (context, index) {
            return ListTile(title: Text(provider.products[index].name));
        },
    ),
);

```

Latihan

1. **Buat App Multi-Screen** — 3 screen: HomeScreen, AboutScreen, ContactScreen. Pake named routes. BottomNavigationBar buat navigasi antar screen. Tiap screen punya AppBar title beda.
2. **Buat API Call ke JSONPlaceholder** — Pake <https://jsonplaceholder.typicode.com/posts>. Model: Post (userId, id, title, body). Tampilkan ListView judul posts. Klik → detail screen pake Navigator push. Tambahin error handling + loading spinner.
3. **Buat Search Screen** — Pake <https://jsonplaceholder.typicode.com/users>. FutureBuilder. Tampilkan ListTile: nama, email, company name. Tambahin TextField buat filter/search by name pake where() di list. Refresh pake RefreshIndicator.
4. **Gabung Provider + API** — Bikin UserProvider extends ChangeNotifier. Method: loadUsers(). UI: pake context.watch<UserProvider>(). Tampilkan loading/error/data. Bonus: tombol retry kalo error.

1.4 Firebase & Deploy ke Play Store

Firebase Setup

1. Buat Project Firebase

1. Buka console.firebase.google.com
2. Klik **Add project** → kasih nama → create
3. Non-aktifin Google Analytics (kecuali butuh)
4. Tunggu provisioning

2. Register App Android

1. Di Firebase Console → project → **Android icon** (+)
2. Isi **Android package name** — cocokin sama android/app/build.gradle
→ applicationId
defaultConfig {
 applicationId "com.example.myapp" *// ini yang lo isi*
 minSdkVersion 21
 targetSdkVersion 34
}
3. Isi **App nickname** (opsional)
4. Debug signing certificate — skip dulu
5. Download google-services.json
6. Letakin di: android/app/google-services.json

3. Update android/build.gradle

```
// android/build.gradle – level project
buildscript {
    dependencies {
        // ... classpath existing
        classpath 'com.google.gms:google-services:4.4.0'
    }
}
```

4. Update android/app/build.gradle

```
// android/app/build.gradle – level app
apply plugin: 'com.google.gms:google-services' // paling bawah

android {
    defaultConfig {
        minSdkVersion 21 // minimal 21 buat Firebase
    }
}
```


5. Install FlutterFire CLI

```
# Install CLI
dart pub global activate flutterfire_cli

# Configure Flutter project
flutterfire configure

# Ini generate:
# - lib/firebase_options.dart
# - Update google-services.json (kalo perlu)
```

6. Init Firebase di main.dart

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const MyApp());
}
```

Tambah dependensi di pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  firebase_core: ^2.24.0
  cloud_firestore: ^4.15.0
  firebase_auth: ^4.16.0
```

Firestore CRUD

Setup Firestore Database

1. Firebase Console → **Firestore Database** → Create database
2. Pilih **Start in test mode** (development)
3. Pilih region (pilih asia-southeast2 — Jakarta)

Model + Service

```
// models/note.dart
class Note {
  final String id;
  final String title;
  final String content;
  final DateTime createdAt;

  Note({
    required this.id,
    required this.title,
    required this.content,
    required this.createdAt,
  });

  // Firestore document → Note
  factory Note.fromFirestore(Map<String, dynamic> data, String docId) {
    return Note(
      id: docId,
      title: data['title'] as String? ?? '',
      content: data['content'] as String? ?? '',
      createdAt: (data['created_at'] as dynamic)?.toDate() ?? DateTime.now(),
    );
  }

  // Note → Map buat Firestore
  Map<String, dynamic> toFirestore() => {
    'title': title,
    'content': content,
    'created_at': Timestamp.fromDate(createdAt),
  };
}

// services/note_service.dart
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/note.dart';

class NoteService {
  final CollectionReference _notes =
    FirebaseFirestore.instance.collection('notes');

  Stream<List<Note>> getNotes() {
    return _notes.orderBy('created_at', descending: true).snapshots().map(
      (snapshot) => snapshot.docs
        .map((doc) => Note.fromFirestore(
```

```

        doc.data() as Map<String, dynamic>,
        doc.id,
    ))
    .toList(),
    );
}

Future<void> addNote(String title, String content) {
    return _notes.add({
        'title': title,
        'content': content,
        'created_at': Timestamp.now(),
    });
}

Future<void> updateNote(String id, String title, String content) {
    return _notes.doc(id).update({
        'title': title,
        'content': content,
    });
}

Future<void> deleteNote(String id) {
    return _notes.doc(id).delete();
}
}

```

UI dengan StreamBuilder

```

class NotesScreen extends StatelessWidget {
    final NoteService _noteService = NoteService();

    NotesScreen({super.key});

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(title: const Text('Catatan')),
            body: StreamBuilder<List<Note>>(
                stream: _noteService.getNotes(),
                builder: (context, snapshot) {
                    if (snapshot.hasError) {
                        return Center(child: Text('Error: ${snapshot.error}'));
                    }
                }
            )
        );
    }
}

```

```

    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(child: CircularProgressIndicator());
    }

    final notes = snapshot.data!;
    if (notes.isEmpty) {
      return const Center(
        child: Text('Belum ada catatan. Tambah sekarang!'),
      );
    }

    return ListView.builder(
      itemCount: notes.length,
      itemBuilder: (context, index) {
        final note = notes[index];
        return Dismissible(
          key: Key(note.id),
          onDismissed: (_) => _noteService.deleteNote(note.id),
          background: Container(color: Colors.red),
          child: ListTile(
            title: Text(note.title),
            subtitle: Text(note.content),
            trailing: Text(
              '${note.createdAt.day}/${note.createdAt.month}',
            ),
          ),
        );
      },
    );

    floatingActionButton: FloatingActionButton(
      onPressed: () => _showAddDialog(context),
      child: const Icon(Icons.add),
    ),
  );
}

void _showAddDialog(BuildContext context) {
  // ... dialog title + content → _noteService.addNote()
}
}

```

Stream vs Future: Firestore pake Stream (real-time). Bedanya ama Future: Stream terus-terusan ngirim data baru. UI auto update tiap ada perubahan di Firestore.

Future cuma sekali.

Firestore Auth

Auth Service

```
// services/auth_service.dart
import 'package:firebase_auth/firebase_auth.dart';

class AuthService {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  // Stream auth state – auto detect login/logout
  Stream<User?> get user => _auth.authStateChanges();

  // Register
  Future<UserCredential> register(String email, String password) async {
    return await _auth.createUserWithEmailAndPassword(
      email: email,
      password: password,
    );
  }

  // Login
  Future<UserCredential> login(String email, String password) async {
    return await _auth.signInWithEmailAndPassword(
      email: email,
      password: password,
    );
  }

  // Logout
  Future<void> logout() async {
    await _auth.signOut();
  }

  // Get current user
  User? get currentUser => _auth.currentUser;
}
```

Login Screen

```
class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});
}
```

```

@override
State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _authService = AuthService();
  bool _isLoading = false;

  Future<void> _login() async {
    setState(() => _isLoading = true);
    try {
      await _authService.login(
        _emailController.text.trim(),
        _passwordController.text,
      );
      // Success → Navigator bakal otomatis pindah
      // karena auth state listener di main
    } on FirebaseAuthException catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(_getErrorMessage(e.code))),
      );
    } finally {
      setState(() => _isLoading = false);
    }
  }

  String _getErrorMessage(String code) {
    switch (code) {
      case 'user-not-found':
        return 'Email tidak terdaftar';
      case 'wrong-password':
        return 'Password salah';
      case 'invalid-credential':
        return 'Email atau password salah';
      case 'too-many-requests':
        return 'Terlalu banyak percobaan. Coba lagi nanti';
      default:
        return 'Login gagal: $code';
    }
  }
}

@override
Widget build(BuildContext context) {

```

```

return Scaffold(
  appBar: AppBar(title: const Text('Login')),
  body: Padding(
    padding: const EdgeInsets.all(24),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        TextField(
          controller: _emailController,
          decoration: const InputDecoration(
            labelText: 'Email',
            prefixIcon: Icon(Icons.email),
            border: OutlineInputBorder(),
          ),
          keyboardType: TextInputType.emailAddress,
        ),
        const SizedBox(height: 16),
        TextField(
          controller: _passwordController,
          decoration: const InputDecoration(
            labelText: 'Password',
            prefixIcon: Icon(Icons.lock),
            border: OutlineInputBorder(),
          ),
          obscureText: true,
        ),
        const SizedBox(height: 24),
        SizedBox(
          width: double.infinity,
          child: ElevatedButton(
            onPressed: _isLoading ? null : _login,
            child: _isLoading
              ? const SizedBox(
                  height: 20,
                  width: 20,
                  child: CircularProgressIndicator(strokeWidth: 2),
                )
              : const Text('Login'),
          ),
        ),
      ],
    ),
  ),
);
}

```

```

@override
void dispose() {
  _emailController.dispose();
  _passwordController.dispose();
  super.dispose();
}
}

```

Auth State Listener — Redirect Otomatis

```

// main.dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: StreamBuilder<User?>(
        stream: FirebaseAuth.instance.authStateChanges(),
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Scaffold(
              body: Center(child: CircularProgressIndicator()),
            );
          }

          // Kalo login → NotesScreen, kalo enggak → LoginScreen
          if (snapshot.hasData) {
            return const NotesScreen();
          } else {
            return const LoginScreen();
          }
        },
      ),
    );
  }
}

```

Firestore Security Rules

```

// Firebase Console → Firestore → Rules

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

```



```

// Cuma user login yang bisa akses
match /notes/{noteId} {
    allow read, write: if request.auth != null
        && request.auth.uid == resource.data.user_id;

    allow create: if request.auth != null
        && request.auth.uid == request.resource.data.user_id;
}
}
}

```

Deploy ke Play Store

Prasyarat

1. **Akun Developer Google Play** — bayar \$25 sekali
2. **Keystore** — file .jks buat sign APK
3. **App sudah stabil** — di-test di device real

1. Generate Keystore

```

# Generate keystore
keytool -genkey -v -keystore ~/upload-keystore.jks \
    -keyalg RSA -keysize 2048 -validity 10000 \
    -alias upload

```

Simpan password! Lo butuh ini.

2. Konfigurasi Signing

Buat android/key.properties:

```

storePassword=passwordkamu
keyPassword=passwordkamu
keyAlias=upload
storeFile=/home/user/upload-keystore.jks

```

Edit android/app/build.gradle:

```

// Di dalam android { ... }
def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}

```

```

android {
    // ... defaultConfig, dll

    signingConfigs {
        release {
            keyAlias keystoreProperties['keyAlias']
            keyPassword keystoreProperties['keyPassword']
            storeFile keystoreProperties['storeFile'] ? file(keystoreProperties[
            storePassword keystoreProperties['storePassword']
        }
    }
    buildTypes {
        release {
            signingConfig signingConfigs.release
        }
    }
}

```

3. Build APK / App Bundle

```

# Clean dulu
flutter clean
flutter pub get

# Build Android App Bundle (.aab) – recommended buat Play Store
flutter build appbundle --release
# Hasil: build/app/outputs/bundle/release/app-release.aab

# Atau APK (buat test internal)
flutter build apk --release
# Hasil: build/app/outputs/flutter-apk/app-release.apk

```

4. Versioning

```

// android/app/build.gradle
android {
    defaultConfig {
        applicationId "com.example.myapp"
        minSdkVersion 21
        targetSdkVersion 34
        versionCode 2 // naikin tiap update
        versionName "1.0.1" // semantic version
    }
}

```

5. Upload ke Play Console

1. Buka play.google.com/console
2. **Buat app** → kasih nama, pilih app/game, pilih free/paid
3. Isi **Store listing**:
 - Deskripsi pendek & panjang
 - Screenshot (min 2, rekomendasi 8)
 - Icon 512×512px
 - Feature graphic 1024×500px
 - Kategori & tags
4. **Production** → **Create new release**
 - Upload .aab
 - Isi release notes (what's new)
5. **Content rating** → isi kuesioner
6. **Target audience & privacy policy**
7. **Pricing & distribution** → pilih negara

Tips:

- Pake **internal testing** dulu — ga langsung publik
 - App pertama: cek dulu nama unik di Play Store
 - Siapin privacy policy — bisa pake generator online
-

Build Flavors (Dev vs Production)

```
// android/app/build.gradle
android {
    flavorDimensions "environment"
    productFlavors {
        dev {
            dimension "environment"
            applicationIdSuffix ".dev"
            versionNameSuffix "-dev"
        }
        prod {
            dimension "environment"
        }
    }
}

// lib/config.dart
class AppConfig {
    static String get apiBaseUrl {
        // Detect flavor dari string constant
        const isDev = String.fromEnvironment('FLUTTER_APP_FLAVOR') == 'dev';
        return isDev ? 'https://dev.api.com' : 'https://api.com';
    }
}
```

```

    }

    static String get projectName {
      const isDev = String.fromEnvironment('FLUTTER_APP_FLAVOR') == 'dev';
      return isDev ? 'MyApp (Dev)' : 'MyApp';
    }
  }

  # Build dengan flavor
  flutter build appbundle --flavor dev --release
  flutter build appbundle --flavor prod --release

  # Run dengan flavor
  flutter run --flavor dev

```

Play Store Checklist

Item	Status
App icon 512×512px	☐
Feature graphic 1024×500px	☐
Screenshot minimal 2 (rekomendasi 8)	☐
Deskripsi pendek max 80 karakter	☐
Deskripsi panjang max 4000 karakter	☐
Privacy policy URL	☐
Content rating selesai	☐
APK/.aab sudah di-sign release	☐
Version code & name di-increment	☐
Test di device real	☐

Latihan

1. **Buat Notes App** — Firebase Firestore CRUD. StreamBuilder + ListView. Tambah note via dialog. Swipe-to-delete pake Dismissible. Pull-to-refresh.
2. **Buat Auth Flow** — Login + Register screen pake Firebase Auth. Kalo user login → HomeScreen. Kalo logout → LoginScreen. Pake authStateChanges() stream. Kasih error handling buat: wrong-password, user-not-found, email-already-in-use, weak-password.

3. **Buat Protected Notes** — Notes cuma bisa dibaca/ditulis sama user yang bikin. Tambah field `user_id` di setiap dokumen. Set Firestore security rules: `request.auth.uid == resource.data.user_id`.
4. **Build APK & Simulasikan Deploy** — Generate keystore, setup signing config, build appbundle release. Print daftar file hasil build. Bikin dummy Google Play Store listing (nama app, deskripsi, screenshot list).